

PUBLISHING AND REFACTORING THE TODOS APP

Created: 2025-12-15 Mon 16:00

CONTENTS

- Publishing the first version of your framework to NPM
- Refactoring the TODOs application to use your framework

BUILDING AND PUBLISHING THE FRAMEWORK

- Export the functions that you want to expose to the developer
 - use `src/index.js` file.
- The `src/index.js` file is the entry point of the build process
 - everything exported will be available in the final bundle.

BUILDING AND PUBLISHING THE FRAMEWORK

- developers need to use:
 - the `h()`, `hString()`, and `hFragment()` functions to create virtual Document Object Model (DOM) nodes
 - `createApp()` function to create an application.
- In the `src/index.js` file:

```
export { createApp } from './app'  
export { h, hFragment, hString } from './h'
```

- Run the `build` script inside the runtime workspace to build the framework:

```
$ npm run build
```

- this script bundles the JavaScript code into a single ESM file:
 - `dist/</fwk-name/>.js`.
 - `</fwk-name/>` corresponds to the `package.json` file's `name` property.
- Also, we must setup an NPM account to publish your package.

- The package should be usable directly from:
 - `packages/runtime/dist/</fwk-name/>.js`

- The package should be usable directly from:
 - `packages/runtime/dist/</fwk-name/>.js`
- alternatively you should be able to install using `npm`

- The package should be usable directly from:
 - `packages/runtime/dist/</fwk-name/>.js`
- alternatively you should be able to install using `npm`

```
$ npm install <fwk-name>
```


- Set the `version` field in `package.json` to `1.0.0`,

- Set the `version` field in `package.json` to `1.0.0`,

```
"version": "1.0.0",
```

- Set the `version` field in `package.json` to `1.0.0`,

```
"version": "1.0.0",
```

- To publish the package to <https://www.npmjs.com>
 - run the `publish` NPM script:

- Set the `version` field in `package.json` to `1.0.0`,

```
"version": "1.0.0",
```

- To publish the package to <https://www.npmjs.com>
 - run the `publish` NPM script:

```
$ npm publish
```

SHORT EXAMPLES

CREATING A COUNTER FOR BUTTON

CREATING A COUNTER FOR BUTTON

- Create a button that counts clicks on it
 - The view is a button, but it's interactive
 - it renders something different based on the state (click count)

```
createApp({  
  state: 0,  
  
  reducers: {  
    add: (state, amount) => state + amount,  
  },  
  
  view: (state, emit) =>  
    h(  
      'button',  
      { on: { click: () => emit('add', 1) } },  
      [hString(state)]  
    ),  
}).mount(document.body)
```


- This code is all you need to make a simple application

- This code is all you need to make a simple application
- No DOM manipulation is involved

- This code is all you need to make a simple application
- No DOM manipulation is involved
- the framework handles that work now.

- This code is all you need to make a simple application
- No DOM manipulation is involved
- the framework handles that work now.
- This application renders as a single button with the number on it

- The html file of the application will be:

```
<body>  
  <button>0</button>  
</body>
```


The framework removes the `<button>` element from the DOM and creates a new one every time the state changes—that is, when you click the button.

The framework removes the `<button>` element from the DOM and creates a new one every time the state changes—that is, when you click the button.

This change happens in a fraction of a millisecond, so you won't even notice it.

MORE COMPLEX EXAMPLE

- Implement a counter application that allows the user to increment and decrement a counter.
- The initial counter is 0, and the user should be able to increment it by clicking a button with a + label.
- The user should also be able to decrement it by clicking another button, this one with the - label.
- The counter should be displayed between the two buttons.

```
<body>  
  <h1>Counter</h1>  
</body>
```

```
<script type="module">
  import {
    createApp,
    h,
    hString,
    hFragment,
  } from 'https://unpkg.com/fe-fwk@1'

  const state = { count: 0 }
  const reducers = {
    add: (state) => ({ count: state.count + 1 }),
    sub: (state) => ({ count: state.count - 1 }),
  }
```

```
function View(state, emit) {
  return hFragment([
    h('button', { on: { click: () => emit('sub') } }, ['-']),
    h(
      'span',
      { class: `${state.count < 0 ? 'negative' : 'positive'}` },
      [hString(state.count)]
    ),
    h('button', { on: { click: () => emit('add') } }, ['+']),
  ])
}

createApp({ state, view: View, reducers }).mount(document.body)
</script>
```

```
<style>
  body {
    text-align: center;
    margin: 2em;
    font-size: 2em;
  }
  button {
    font-size: 1em;
    width: 1.5em;
  }
```

```
span {  
  margin: 0 1em;  
}  
span.negative {  
  color: #e91e61;  
}  
</style>
```

REFACTORING THE TODOS APP

REFACTORING THE TODOS APP

- Rewrite the original TODOs app using the framework

REFACTORING THE TODOS APP

- Rewrite the original TODOs app using the framework
- Simpler code

REFACTORING THE TODOS APP

- Rewrite the original TODOs app using the framework
- Simpler code
- The virtual DOMs must be written instead of HTML

REFACTORING THE TODOS APP

- Rewrite the original TODOs app using the framework
- Simpler code
- The virtual DOMs must be written instead of HTML
- See the benefits of using the newly developed front-end framework

REFACTORING THE HTML FILE

REFACTORING THE HTML FILE

- Clean up the `<body>` tag in `todos.html` (it should be empty)

REFACTORING THE HTML FILE

- Clean up the `<body>` tag in `todos.html` (it should be empty)

```
<!-- FILE: todos.html -->
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <script type="module" src="todos.js"></script>
    <title>My TODOs</title>
  </head>
  <body></body> <!-- The body is empty now -->
</html>
```


REFACTORING THE `todo.js`

REFACTORING THE `todo.js`

- Write the code from scratch

REFACTORING THE `todo.js`

- Write the code from scratch
- Import functions exported by the framework from the `dist` directory

REFACTORING THE `todo.js`

- Write the code from scratch
- Import functions exported by the framework from the `dist` directory

```
// FILE: todo.js
import { createApp, h, hFragment } from 'https://unpkg.com/<fwk-name>
```


DEFINING THE APPLICATION'S STATE

- Previously the state of the TODOs application was an array of strings

```
const addTodoInput = document.getElementById('todo-input')  
// The text of the new to-do item  
addTodoInput.value
```

- Abstract away the manipulation of the DOM
- Any relevant information should be part of the state.
- Need to keep three pieces of information in the state:

DEFINING THE APPLICATION'S STATE

DEFINING THE APPLICATION'S STATE

todos

DEFINING THE APPLICATION'S STATE

todos

The array of to-do items

DEFINING THE APPLICATION'S STATE

todos

The array of to-do items

currentTodo

DEFINING THE APPLICATION'S STATE

todos

The array of to-do items

currentTodo

The text of the new to-do item that the user is typing in the input field

edit

An object containing information about the to-do item being edited:

edit

An object containing information about the to-do item being edited:

idx

edit

An object containing information about the to-do item being edited:

idx

The index of the to-do item in the `todos` array that's edited

edit

An object containing information about the to-do item being edited:

idx

The index of the to-do item in the `todos` array that's edited

original

edit

An object containing information about the to-do item being edited:

idx

The index of the to-do item in the `todos` array that's edited

original

The original text of the to-do item before the user started editing

edit

An object containing information about the to-do item being edited:

idx

The index of the to-do item in the `todos` array that's edited

original

The original text of the to-do item before the user started editing

edited

edit

An object containing information about the to-do item being edited:

idx

The index of the to-do item in the `todos` array that's edited

original

The original text of the to-do item before the user started editing

edited

The text of the to-do item as the user is editing it

```
import { createApp, h, hFragment
      } from 'https://unpkg.com/<fwk-name>@1'

const state = {
  currentTodo: '',
  edit: {
    idx: null,
    original: null,
    edited: null,
  },
  todos: ['Walk the dog', 'Water the plants'],
}
```

DEFINING THE REDUCERS

Update the current to-do.

The user types a new character in the input field, so the current to-do needs to be updated.

Add a new to-do.

The user clicks the Add button to add a new to-do to the list.

Start editing a to-do.

The user double-clicks a to-do item to start editing it.

Edit a to-do.

The user types a new character in the input field while editing a to-do item.

Save an edited to-do.

The user finishes editing a to-do and saves the changes.

Cancel editing a to-do.

The user cancels editing a to-do and discards the changes.

Remove a to-do.

The user marks a to-do as completed so that it can be removed from the list.

THE REDUCER FUNCTIONS OF THE STATE (TODOS.JS)

```
const reducers = {  
  // The reducer receives the current TODO as payload  
  'update-current-todo': (state, currentTodo) => ({  
    ...state,  
    currentTodo, // Updates the current TODO in the state  
  }),  
  'add-todo': (state) => ({  
    ...state,  
    // Sets and empty string as the current TODO (clear it)  
    currentTodo: '',  
    // Adds the new TODO to the list  
    todos: [...state.todos, state.currentTodo],  
  }),  
}
```

```
// The reducer receives the index of the TODO to edit as payload
'start-editing-todo': (state, idx) => ({
  ...state,
  edit: {
    idx,
    // Saves the original TODO in case the edition is canceled
    original: state.todos[idx],
    // The edited version of the TODO is the current TODO.
    edited: state.todos[idx],
  },
}),
// The reducer receives the edited TODO as payload.
'edit-todo': (state, edited) => ({
  ...state,
  // Updates the edited TODO in the state
  edit: { ...state.edit, edited },
}),
```

```
'save-edited-todo': (state) => {  
  // Copies the TODOs array  
  const todos = [...state.todos]  
  
  // Replaces the edited TODO in the TODOs array  
  todos[state.edit.idx] = state.edit.edited  
  return {  
    ...state,  
  
    // Resets the edit TODO part of the state  
    edit: { idx: null, original: null, edited: null },  
    todos,  
  }  
},
```

```
'cancel-editing-todo': (state) => ({
  ...state,

  // Resets the edit TODO part of the state
  edit: { idx: null, original: null, edited: null },
}),

// The reducer receives the index of the TODO to remove as payload
'remove-todo': (state, idx) => ({
  ...state,

  // Filters out the TODO with the given index
  todos: state.todos.filter((_, i) => i !== idx),
}),
}
```

- events like 'add-todo' don't have a payload:

```
'add-todo': (state) => ({  
  ...state,  
  currentTodo: '',  
  todos: [...state.todos, state.currentTodo],  
})
```

DEFINING THE VIEW

- Split the application into components
 - top-level component, which we'll call `App ( )`
 - It's a fragment containing:
 - the title (`<h1>` element)
 - `CreateTodo ( )` component,
 - `TodoList ( )` component.

```
function App(state, emit) {  
  return hFragment([  
    h('h1', {}, ['My TODOs']),  
    CreateTodo(state, emit),  
    TodoList(state, emit),  
  ])  
}
```

Let's create the `CreateTodo()` component

- **Recall:** the components are now functions that take in not only the state, but also the `emit()` function to dispatch events to the application's dispatcher.

```
function CreateTodo({ currentTodo }, emit) {  
  // Destructures the currentTodo from the state object  
  return h('div', {}, [  
    // The input's label  
    h('label', { for: 'todo-input' }, ['New TODO']),  
    // The input field's value is the currentTodo in the state.  
    h('input', {  
      type: 'text',  
      id: 'todo-input',  
      value: currentTodo,  
    })  
  ])  
}
```

```

on: {
  input: ({ target }) =>
    // Updates the field's value when the user types in it
    emit('update-current-todo', target.value),
  keydown: ({ key }) => {
    // Checks whether the user pressed the Enter key
    // and the input field has at least three characters
    if (key === 'Enter' && currentTodo.length >= 3) {
      // Dispatches the 'add-todo' command
      emit('add-todo')
    }
  },
},
},
}),

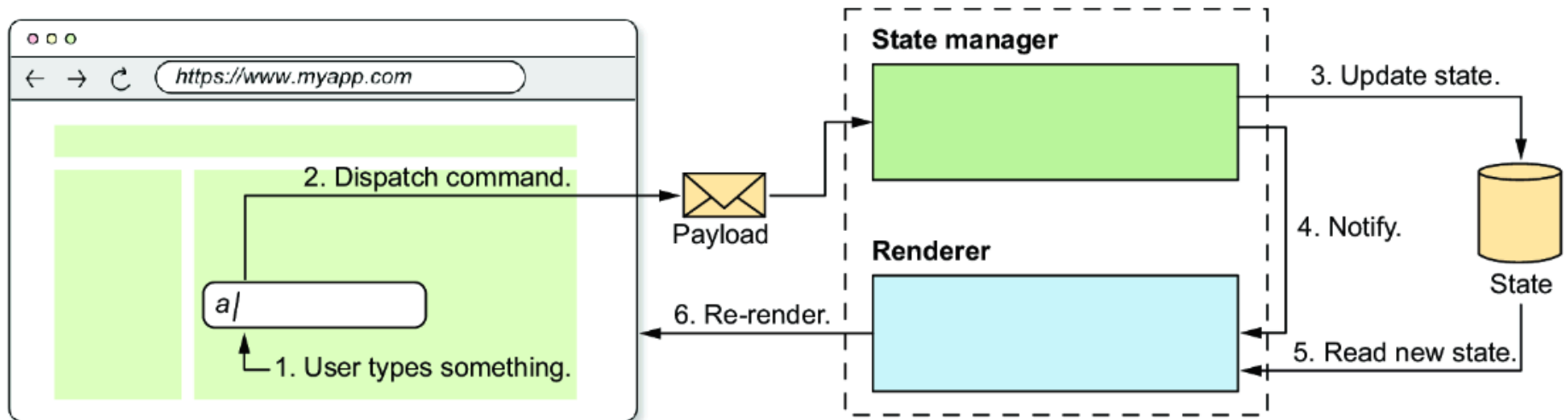
```

```
h(  
  'button',  
  {  
    // Disables the button if the input field has  
    // fewer than three characters  
    disabled: currentTodo.length < 3,  
  
    // Dispatches the 'add-todo' command when the user  
    // clicks the button  
    on: { click: () => emit('add-todo') },  
  },  
  ['Add']  
)  
])  
}
```

- The `emit()` function is used inside a component
- The case of the `<input>` is interesting:
 - two-way binding between the input field and the state.

A two-way binding reflects the changes in the state in the input field, and anything typed in the input field is set in the state and anything typed in the input field is set in the state.

```
h('input', {  
  type: 'text',  
  value: state.currentTodo,  
  on: {  
    input: ({ target }) => emit('update-current-todo', target.value)  
  }  
})
```



Implement the `TodoList()` component

```
function TodoList({ todos, edit }, emit) {  
  return h(  
    'ul',  
    {},  
    todos.map((todo, i) => TodoItem({ todo, i, edit }, emit))  
  )  
}
```

- The `TodoList()` component is a simple one:
 - a `<ul>` element with a list of `TodoItem()` components.

The TodoItem() component

```
function TodoItem({ todo, i, edit }, emit) {  
  const isEditing = edit.idx === i  
  
  return isEditing  
    ? h('li', {}, [ // The item in edit mode  
      h('input', {  
        // The input field's value is the edited TODO's description  
        value: edit.edited,  
        on: {  
          // Updates the field's value when the user types in it  
          input: ({ target }) => emit('edit-todo', target.value)  
        },  
      }),  
    ],  
  ),  
}
```

```
h(  
  'button',  
  {  
    on: {  
      // Saves the edited TODO by dispatching the  
      // 'save-edited-todo' command  
      click: () => emit('save-edited-todo')  
    }  
  },  
  ['Save']  
)
```

```
h(  
  'button',  
  {  
    on: {  
      // Cancels the edit mode by dispatching the  
      // 'cancel-editing-todo' command  
      click: () => emit('cancel-editing-todo')  
    }  
  },  
  ['Cancel']  
)  
])
```

```
: h('li', {}, [  
  // The item in read mode  
  h(  
    'span',  
    {  
      on: {  
        // Starts editing the TODO by dispatching the  
        // 'start-editing-todo' command  
        dblclick: () => emit('start-editing-todo', i)  
      }  
    },  
    // The TODO's description coming from the state  
    [todo]  
  ),
```

```
h(  
  'button',  
  {  
    on: {  
      // Removes the TODO by dispatching the  
      // 'remove-todo' command  
      click: () => emit('remove-todo', i)  
    }  
  },  
  ['Done']  
)  
])  
}
```

- Passing the to-do item's index because some of the dispatched events need to know it
- That index is known by the parent component, `TodoList()`,
 - needs to pass it to the child component as a prop.
- This index isn't part of the application's state.

Putting everything together.

```
createApp({ state, reducers, view: App }).mount(document.body)
```

```
$ npm run serve:examples
```

SUMMARY

SUMMARY

- To publish your framework on NPM, first bundle it by using `npm run build`; then publish it by using the `npm publish` command.

SUMMARY

- To publish your framework on NPM, first bundle it by using `npm run build`; then publish it by using the `npm publish` command.
- Whatever you export from the `src/index.js` file is what's going to be available to the users of your framework.

SUMMARY

- To publish your framework on NPM, first bundle it by using `npm run build`; then publish it by using the `npm publish` command.
- Whatever you export from the `src/index.js` file is what's going to be available to the users of your framework.
- In a two-way binding, the state and the DOM are synchronized. Whatever changes on one side is reflected on the other.